

Hardened Same-Model Evaluation of a Native Multi-Agent Runtime

FallSoftCo Losangelex Research Note

Evidence frozen: 2026-05-22, America/New_York

Abstract

We evaluate whether the Losangelex/Hollywood native multi-agent runtime provides empirical value over same-model Codex agent baselines on published or reproducible agent benchmarks. The study separates four effects that are often conflated: ordinary serial Codex cohorts, true Codex subagent orchestration, native Losangelex rooms, and a single-agent full-context ceiling. On Silo-Bench $n=5$ and $n=10$ observed matrices, true Codex subagents achieve the best cohort strict correctness (24/30 and 23/30), while Losangelex is substantially faster (113.0s and 133.6s mean runtime) and records zero coordination-router errors; the full-context Codex oracle remains stronger and faster than both cohort designs. On the locked native MARBLE database final-primary holdout, Losangelex and serial Codex recover all gold root-cause labels on 30/30 tasks, while Codex subagents recover all labels on 29/30. Codex subagents have slightly higher precision/F1 (0.617/0.742) than Losangelex (0.594/0.740), but Losangelex has lower mean runtime (160.2s vs. 184.0s) and zero router errors (0 vs. 25). The current evidence therefore does not support a universal multi-agent SOTA claim. It supports a narrower systems claim: native Losangelex rooms can preserve or improve benchmark-aligned recall versus same-model agent cohorts while reducing wall-clock latency and coordination failure surface, but full-context single-agent execution remains a critical ceiling control and benchmark-general claims require larger Silo and SWE-bench evidence.

1 Question

The empirical question is not whether “more agents” can be made to do more work. It is whether a durable native multi-agent runtime gives measurable value over the same model operating as ordinary Codex agents or Codex subagents. We therefore report both positive and negative results. A useful outcome is not necessarily a leaderboard win; it may be a reliable latency or failure-mode improvement under equal task scoring.

The current benchmark set has three roles. Silo-Bench tests private-shard distributed coordination. MARBLE/MultiAgentBench database diagnosis tests live diagnostic reasoning against a PostgreSQL substrate. SWE-bench Lite is a negative control for small repair tasks where a single capable agent should often be

enough.

2 Systems

Serial Codex cohort. For Silo and MARBLE, the serial Codex baseline runs one non-interactive Codex execution per benchmark agent profile or shard and coordinates through benchmark-visible files. It is not a native subagent tree.

Codex subagents. The Codex-subagents baseline uses real `spawn_agent` and `wait_agent` calls under the same model family. The parent prepares worker prompts, waits for outputs, and emits scored artifacts. This is the closest same-product comparison to Losangelex native rooms, but it is not parent-blind: the parent has to pass the private worker prompts into the subagents.

Losangelex/Hollywood. Losangelex is evaluated through native Hollywood rooms on the Codex app-server path, with persistent room state, agent identities, coordination tools, heartbeats, and managed app-server startup.

Full-context oracle. For Silo, a single Codex execution is given all private shards for a task and writes the same per-agent output files. This is not a deployable privacy-preserving multi-agent system. It is a ceiling control for the question, “does coordination add value beyond simply putting all context in one prompt?”

3 Validity Controls

The Silo and MARBLE runners use campaign-local `CODEX_HOME` directories and managed app-server startup. Published benchmark repositories, answer keys, and previous result directories are hidden from model workspaces during model-in-the-loop execution. Result JSON records hidden paths, app-server metadata, model settings, raw outputs, deterministic scores, elapsed time, and coordination-tool summaries.

The native MARBLE runner starts the published Docker/PostgreSQL database environment with campaign-local compose overrides. Agents receive a workspace containing case observations, database notes, and a `query_db.py` client for live PostgreSQL inspection. Gold root-cause labels remain outside the workspace and are used only after execution for deterministic scoring.

Table 1: Silo-Bench scale context. Losangelex rows are hidden-path native-runtime runs with GPT-5.4; public rows are Silo-Bench paper baselines.

System/model	n=2	n=5	n=10	n=20
DeepSeek-V3.1	61.2%	48.5%	39.9%	33.6%
GPT-OSS-120B	34.4%	28.0%	14.0%	13.2%
Qwen3-Next-80B	17.2%	9.1%	8.6%	7.4%
Losangelex+GPT-5.4	73.3%	70.0%	66.7%	63.3%

Post-run scans check for answer-key leakage, published-repository path access, shell fallback strings, permission failures, and `SPAWN_FAILED` markers. The final-primary MARBLE run reported below had a clean scan and left no MARBLE/PostgreSQL containers running.

4 Silo-Bench

Silo-Bench is the clearest distributed-coordination benchmark in the current evidence base. Each task gives agents private information and scores whether their final submissions collectively solve a global problem. We keep strict Silo scoring as primary. Numeric-tolerance audit fields are useful for diagnosing exact-format failures, but they do not replace the published strict metrics.

4.1 Public Scale Context

Losangelex hidden-path matrices are complete for $n=2$, $n=5$, $n=10$, and $n=20$. Table 1 compares those strict success rates against the strongest public Silo-Bench paper baselines available in our ledger. This table is useful context, not an official leaderboard claim: the compared studies use different models and harnesses.

4.2 Same-Model Controls

Table 2 is the more important Silo result because it compares same-model baselines. It changes the interpretation materially. True Codex subagents beat Losangelex on strict success count at $n=5$ and $n=10$, while Losangelex is faster and cleaner operationally. The full-context oracle is more accurate and faster than all cohort systems, sharply limiting any simple “multi-agent beats single-agent” claim.

The $n=5$ Codex-subagent artifact also reports numeric-tolerance success of 28/30 after recursive tolerance scoring was added. This does not change strict scoring; it shows that several Silo misses are exact-format failures rather than semantic misses. The paper therefore treats Silo as evidence for a constrained tradeoff: Codex subagents are the stronger observed cohort on strict Silo correctness, Losangelex is the lower-latency native-room baseline with fewer coordination failures, and full-context Codex remains the ceiling when privacy/distribution constraints are removed.

5 Native MARBLE Database

The locked final-primary MARBLE holdout is the strongest native non-Silo result. It uses 30 database tasks selected before final evaluation and runs serial Codex, true Codex subagents, and Losangelex against the same native PostgreSQL/Docker substrate. Table 3 gives the aggregate.

Losangelex is faster than serial Codex on 30/30 paired tasks and faster than Codex subagents on 23/30. Mean paired runtime ratios are 2.51x for serial Codex over Losangelex and 1.17x for Codex subagents over Losangelex. The only non-full-recall result is Codex subagents on `database-091`, which predicted `REDUNDANT_INDEX` against gold `REDUNDANT_INDEX, VACUUM`.

The result should be read with the MARBLE scoring semantics in mind. MARBLE’s database evaluator emphasizes recall over gold root-cause labels, while exact-set match and precision/F1 are stricter auxiliary diagnostics. On that primary recall-oriented view, Losangelex has the best heldout result jointly with serial Codex and ahead of Codex subagents by one task. On precision/F1, Codex subagents are slightly ahead. The defensible claim is therefore not “Losangelex dominates MARBLE.” It is that Losangelex preserves full recall while improving latency and avoiding observed subagent router errors.

The development evidence points in the same direction. Across a 40-task native MARBLE development table, serial Codex reached 39/40 full-recall successes, Codex subagents 40/40, and Losangelex 40/40. Codex subagents and Losangelex tied average precision/F1 at 0.583/0.733, while Losangelex was faster than Codex subagents on 39/40 tasks and recorded zero coordination-tool errors versus 44 router errors for Codex subagents.

6 SWE-bench Negative Control

SWE-bench Lite remains a small negative control rather than a main claim. On three official instances, both Codex and Losangelex resolved all three, but Losangelex was slower: 189.0s mean runtime versus 70.8s. This is expected for small repair tasks where native room overhead does not buy additional information flow. The result argues against presenting Losangelex as universally preferable.

7 Interpretation

The strongest current outcome is a systems result, not a universal SOTA result. Silo shows that decomposition strategy matters: true Codex subagents can improve strict correctness over both serial Codex and Losangelex on observed $n=5/n=10$ matrices, but they do so with higher latency than Losangelex and many more

Table 2: Same-model Silo-Bench controls. Strict S/P are the published-compatible metrics. Router errors count benchmark-observed coordination-tool errors.

Scale	System	Strict successes	Avg S	Avg P	Mean time	Router errors
n=5	Serial Codex cohort	20/30	0.700	0.714	422.1s	0
n=5	Codex subagents	24/30	0.800	0.813	240.1s	70
n=5	Losangelex rooms	21/30	0.807	0.826	113.0s	0
n=5	Full-context Codex oracle	25/30	0.833	0.846	39.2s	0
n=10	Serial Codex cohort	21/30	0.727	0.780	835.5s	0
n=10	Codex subagents	23/30	0.767	0.780	328.6s	182
n=10	Losangelex rooms	20/30	0.733	0.779	133.6s	0
n=10	Full-context Codex oracle	24/30	0.800	0.814	52.8s	0

Table 3: Native MARBLE database final-primary holdout, 30 tasks. Full success means all gold root-cause labels were recovered.

System	Full	Exact	Recall	Prec.	F1	Time	Errors
Serial Codex	30/30	0/30	1.000	0.583	0.733	393.7s	0
Codex subagents	29/30	2/30	0.983	0.617	0.742	184.0s	25
Losangelex	30/30	1/30	1.000	0.594	0.740	160.2s	0

Table 4: SWE-bench Lite negative-control slice.

System	Instances	Resolved	Mean time
Codex	3	3	70.8s
Losangelex	3	3	189.0s

coordination-router errors. Full-context Codex does better still, so when the task allows all private information to be placed in one prompt, coordination is not the best known strategy.

MARBLE gives the cleanest heldout native-runtime evidence. Losangelex reaches full recall on every final-primary task, avoids the Codex-subagent recall miss, and runs faster on average than both baselines except that Codex subagents retain a small precision/F1 advantage. This makes the practical claim narrower and more useful: Losangelex can reduce latency and coordination failure surface while preserving recall on a live diagnostic benchmark.

8 Threats to Validity

Silo n=5/n=10 Codex-subagent runs are observed-development evidence, not a final heldout Silo n=50 study. Several Silo failures are exact-output formatting issues under strict scoring, so numeric-tolerance audits should be reported as auxiliary diagnostics only. The full-context oracle is not privacy preserving, but it is essential for measuring whether coordination itself adds value. MARBLE final-primary is held out for this frozen candidate, but it is still 30 database tasks rather than the full 100-task database set. SWE-bench is only a three-instance negative control and should be expanded to multi-file, dependency-chain, and verifier-heavy issues before making software-repair claims. Runtime numbers include harness differences and should be interpreted as measured wall-clock under this benchmark setup, not as

universal deployment latency.

9 Conclusion

The evidence rejects a simplistic story. Losangelex is not universally better than Codex subagents, and full-context Codex is a strong ceiling where privacy and distribution constraints are absent. The evidence does support a narrower and publishable systems claim: on native MARBLE final-primary, Losangelex preserves full diagnostic recall while reducing mean runtime and coordination errors relative to true Codex subagents; on Silo, Losangelex is a lower-latency native-room baseline, while Codex subagents are currently stronger on strict observed n=5/n=10 correctness and full-context Codex is the ceiling. The next paper-grade expansion is heldout Silo n=50 and a larger stratified SWE-bench sample.

Artifacts

Primary artifacts are:

- `tmp/research/published-agent-benchmarks/marble-native-final-primary-all-systems-2026-05-22/results.json`
- `tmp/research/published-agent-benchmarks/silo-codex-subagents-n5-full-observed-2026-05-21/results.json`
- `tmp/research/published-agent-benchmarks/silo-codex-subagents-n10-full-observed-2026-05-21/results.json`
- `tmp/research/published-agent-benchmarks/silo-managed-hidden-n5-full-losangelex-2026-05-20/results.json`
- `tmp/research/published-agent-benchmarks/silo-managed-hidden-n10-full-losangelex-2026-05-20/results.json`
- `tmp/research/published-agent-benchmarks/silo-full-context-oracle-n5-2026-05-21/results.json`

- `tmp/research/published-agent-benchmarks/silo-full-context-oracle-n10-2026-05-21/results.json`

References

- [1] Jimenez et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? <https://arxiv.org/abs/2310.06770>
- [2] Zhang et al. Silo-Bench: A Scalable Environment for Evaluating Distributed Coordination in Multi-Agent LLM Systems. <https://arxiv.org/abs/2603.01045>
- [3] Zhu et al. MultiAgentBench: Evaluating the Collaboration and Competition of LLM Agents. <https://arxiv.org/abs/2503.01935>
- [4] Liu et al. AgentBench: Evaluating LLMs as Agents. <https://arxiv.org/abs/2308.03688>
- [5] Wu et al. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. <https://arxiv.org/abs/2308.08155>